

AI

AI use and design judgment

AI tools supported architecture brainstorming, implementation, debugging, test verification, and documentation. The submitted design remains the student's responsibility; the decisions below are stated in terms that can be explained from the source code.

AI tools used

Tool	How it was used
Claude by Anthropic	Proposed the initial package split, generated the first implementation and documentation drafts, exercised example scenarios, and helped diagnose a Java classpath issue in PowerShell.
ChatGPT Work with Codex by OpenAI	Reviewed the assignment and source, recompiled the project, ran a deterministic 12-test suite plus a Driver scenario, and rewrote the Design, Tests, and AI documents for clarity and layout.

Three important AI interactions

1 Architecture and implementation

The assignment was provided to Claude, which proposed a reusable assignment2.core package and a Mastermind-specific assignment2.mastermind package. The proposal separated secret generation, validation, evaluation, feedback, history, and user interaction behind small contracts, then generated the initial Java implementation. The final source retains this split because each interface maps to a distinct responsibility used by GuessingGameEngine.

2 Build and classpath debugging

When java assignment2.Driver mastermind failed from a VS Code PowerShell terminal, Claude identified two causes: the Bash-only \$(find ...) build command did not work in PowerShell, and the compiled output directory was not on the Java classpath. The corrected approach compiles into an output folder and either runs with -cp out or runs from that folder.

3 Verification and document redesign

ChatGPT with Codex checked the final source against the assignment, compiled it through the JDK compiler module, and ran 12 deterministic tests. It then replaced the original dense documents with a routed architecture diagram, responsibility matrices, requirement coverage, concise test tables, and this AI-use summary.

AI suggestion reviewed and rejected

Suggestion

Claude AI suggested creating a separate Command interface and individual command classes for HISTORY and REVEAL instead of handling those commands directly in GuessingGameEngine.

Decision

I rejected this suggestion for Phase I because the program currently has only two simple commands, and the existing code handles them clearly with two if statements. Two commands are not complicated enough to justify several additional classes. Adding a Command interface and several new classes would make the design harder to follow without providing much benefit. If Phase II introduces many commands or allows different games to define their own commands, separate command classes may become useful and I would reconsider the approach.

Criterion	Evaluation
Current complexity	Only HISTORY and test-only REVEAL are recognized.
Benefit of abstraction now	Small; it would move two direct comparisons into several new types.
Cost	More indirection, construction logic, and explanation during checkout.
Revisit when	Commands vary by game, become numerous, or need dynamic registration.

Responsibility for the submission

AI output was treated as a draft to inspect rather than as an authority. The final architecture can be defended from the code: the engine depends on interfaces, Mastermind owns its scoring and feedback rules, invalid guesses do not consume attempts, and deterministic secret generation makes tests repeatable. The student remains responsible for explaining, critiquing, and extending every submitted line.